



# ADVANCED DATABASE SYSTEMS

WEEK 3

# QUERY PROCESSING PART II



# Cost-based Query Subsystem

Queries

```
Select *  
From Blah B  
Where B.blah = 'blah'
```

Query Parser

Query Optimizer

Plan  
Generator

Plan Cost  
Estimator

Query Plan Evaluator

Usually there is a  
heuristics-based  
rewriting step before  
the cost-based steps.

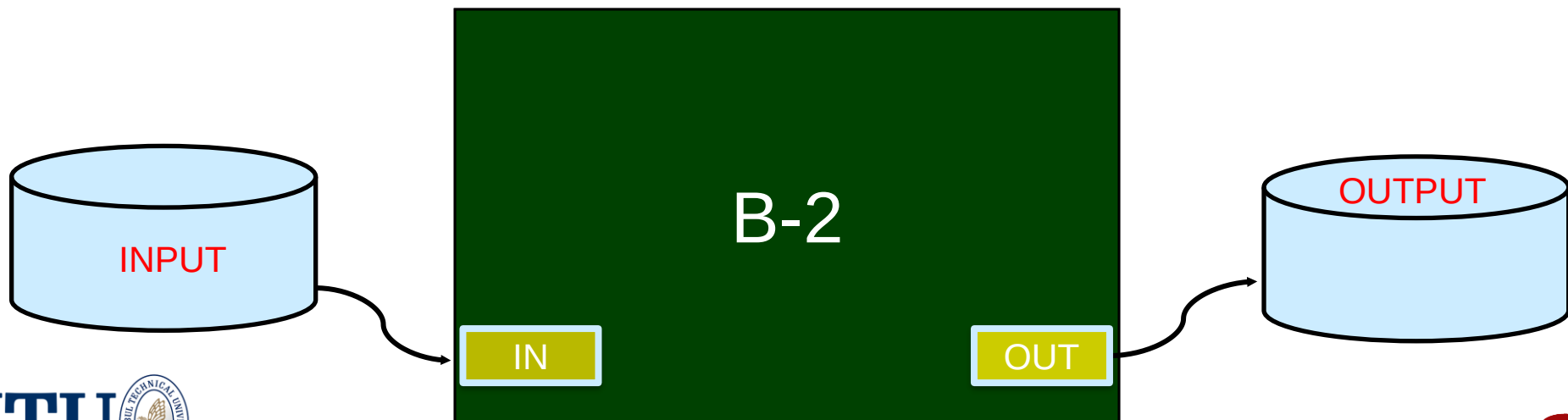
Catalog Manager

Schema

Statistics

# Divide and Conquer

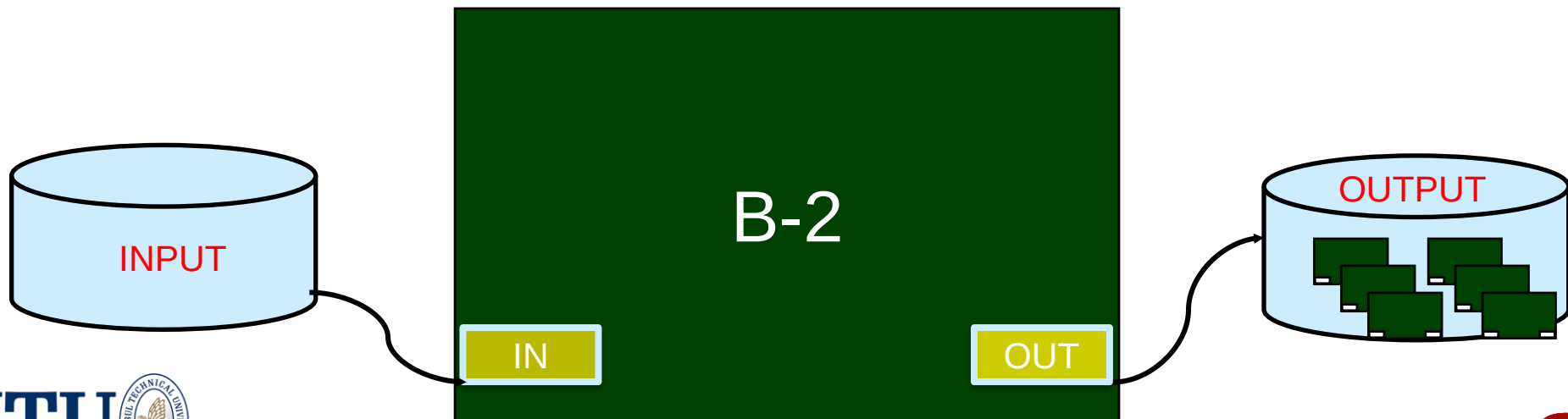
- *Out-of-core* algorithms orchestrate rendezvous.
- Typical RAM Allocation:
  - Assume  $B$  pages worth of RAM available
  - Use 1 page of RAM to read into (input buffer)
  - Use 1 page of RAM to write into (output buffer)
  - $B-2$  pages of RAM as workspace



# Divide and Conquer

## □ Phase 1

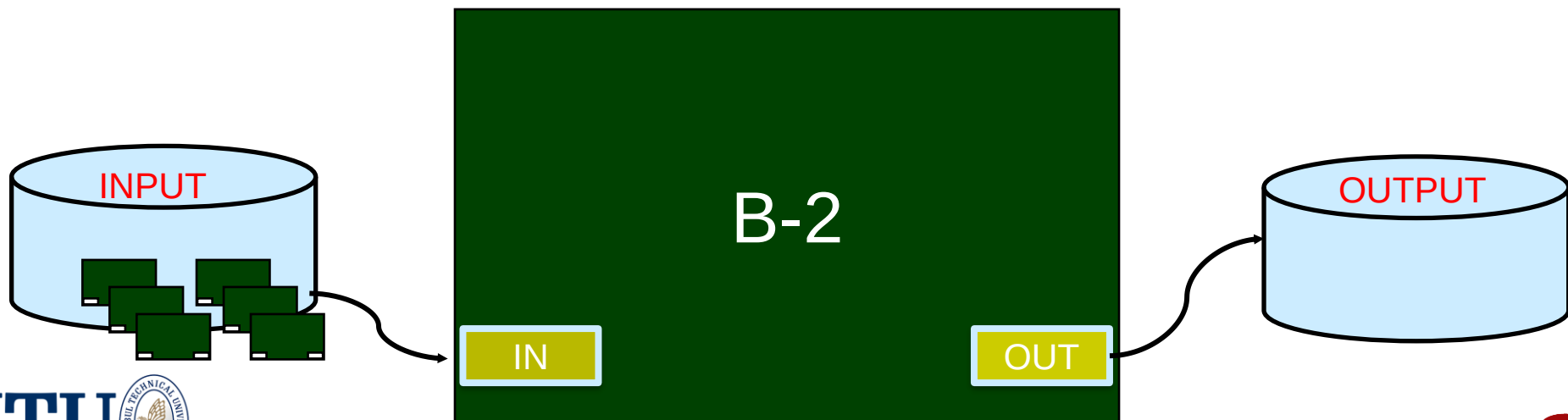
- “streamwise” *divide* into  $N/(B-2)$  megachunks
- output (write) to disk one megachunk at a time



# Divide and Conquer

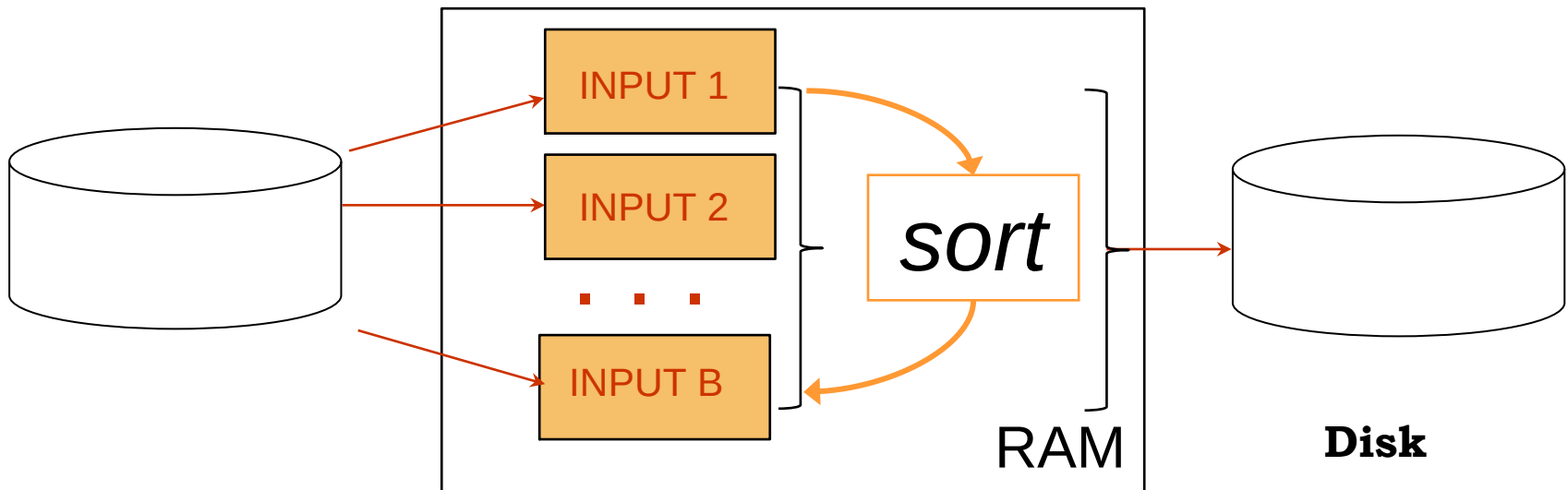
## □ Phase 2

- Now megachunks will be the input
- Process each *megachunk* individually.



# General External Merge Sort

- More than 3 buffer pages. How can we utilize them?
- To sort a file with  $N$  pages using  $B$  buffer pages:
  - Pass 0: use  $B$  buffer pages. Produce  $\lceil N / B \rceil$  sorted runs of  $B$  pages each.

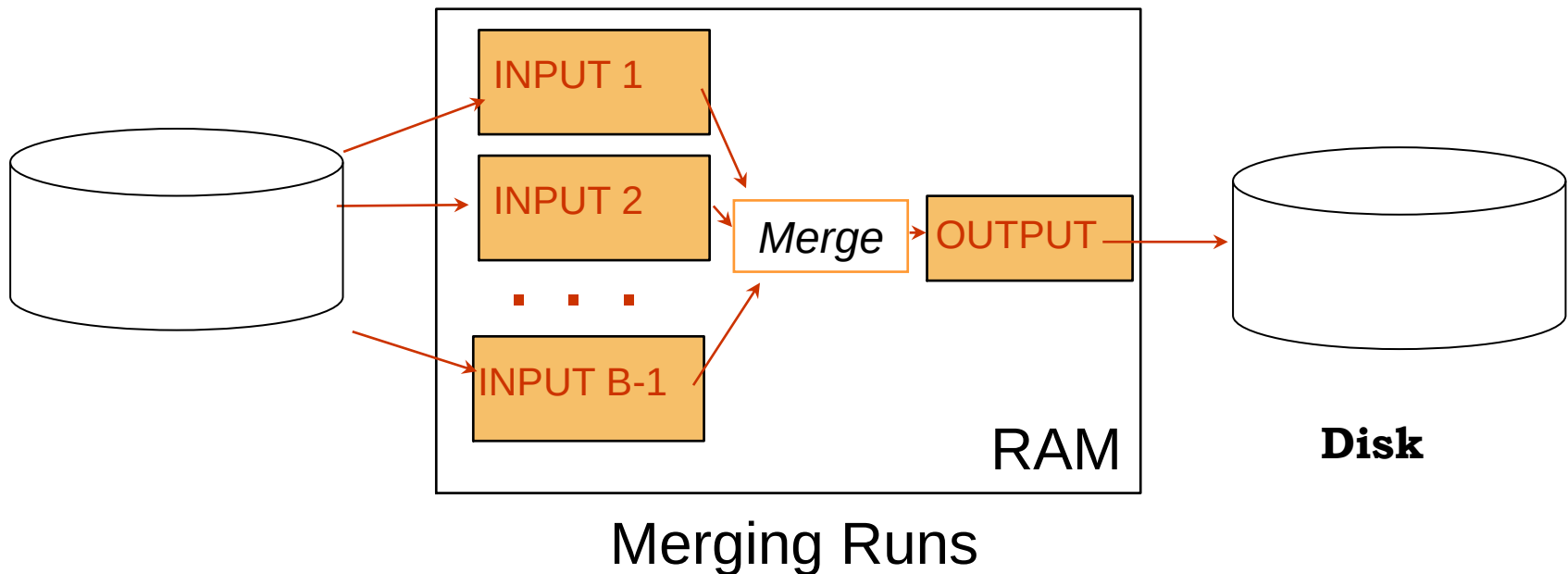


Pass 0 – Create Sorted Runs

# General External Merge Sort

Pass 1, 2, ..., etc.: merge B-1 runs.

Creates runs of  $(B-1) * \text{size of runs from previous pass (i.e., B)}$ .



# Two-Way External Merge Sort

## Sort subfiles and Merge

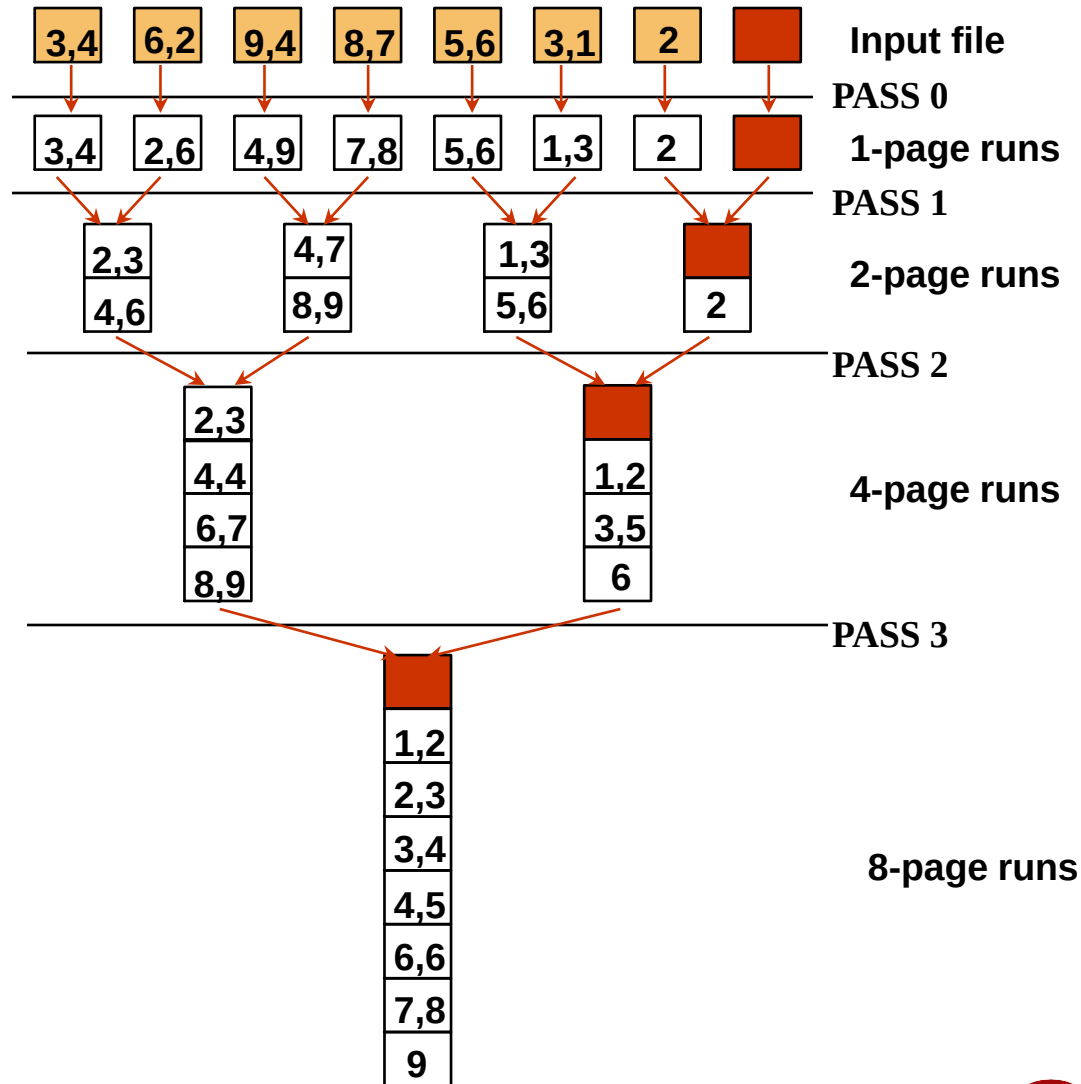
- How many passes?
- N pages in the file  
=> the number of passes =

$$\lceil \log_2 N \rceil + 1$$

- Total I/O cost? (reads + writes)
- Each pass we read + write each page in file.

So total cost is:

$$2N(\lceil \log_2 N \rceil + 1)$$



# Alternative to Sorting: Hashing

- Idea: Many times we don't require order
  - e.g.: removing duplicates
  - e.g.: forming groups
- Often just need to *bring together* matches
- Hashing does this
  - And may be cheaper than sorting! (Hmmm...!)
  - But how to do it??



# Divide & Conquer

- Streaming Partition (divide):  
Use a hash function  $h_p$  to stream records to **disk-based** partitions
  - All matches fall into the same partition.
  - *Streaming* all to create partitions on disk:
    - ▶ “Spill” partitions to disk via output buffers



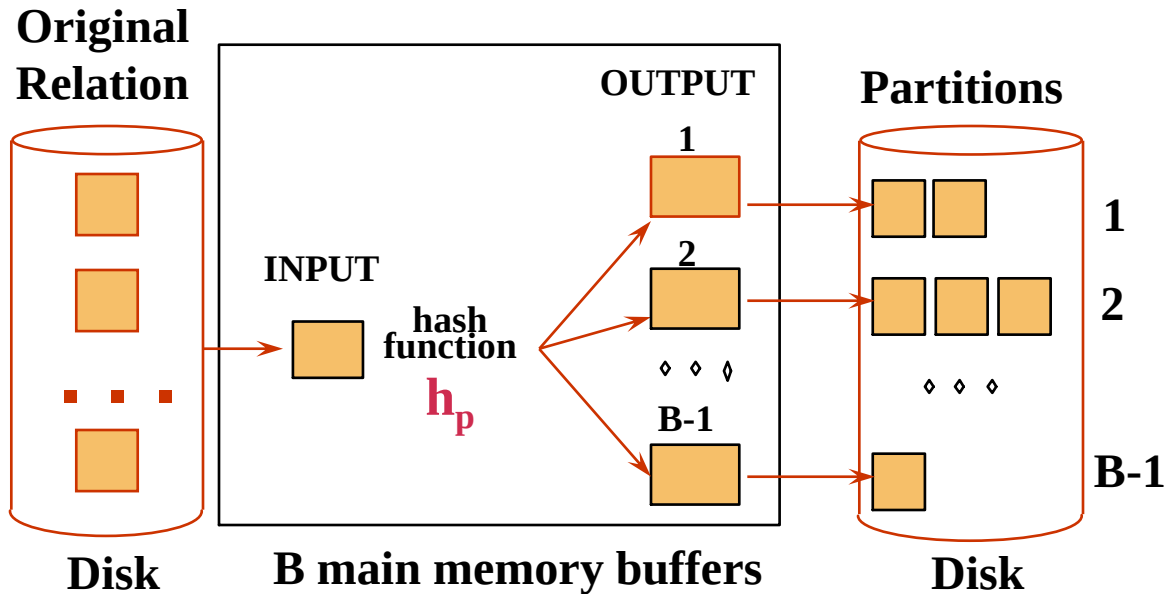
# Divide & Conquer

- Streaming Partition (divide):  
Use a hash function  $h_p$  to stream records to **disk-based** partitions
  - All matches fall into the same partition.
  - *Streaming* all to create partitions on disk:
    - ▶ “Spill” partitions to disk via output buffers
- ReHash (conquer):  
Read partitions into **RAM-based** hash table one at a time, using hash function  $h_r$ 
  - Then go through each bucket of this hash table to locate matchings in RAM
- Note: Two different hash functions
  - ▶  $h_p$  is coarser-grained than  $h_r$

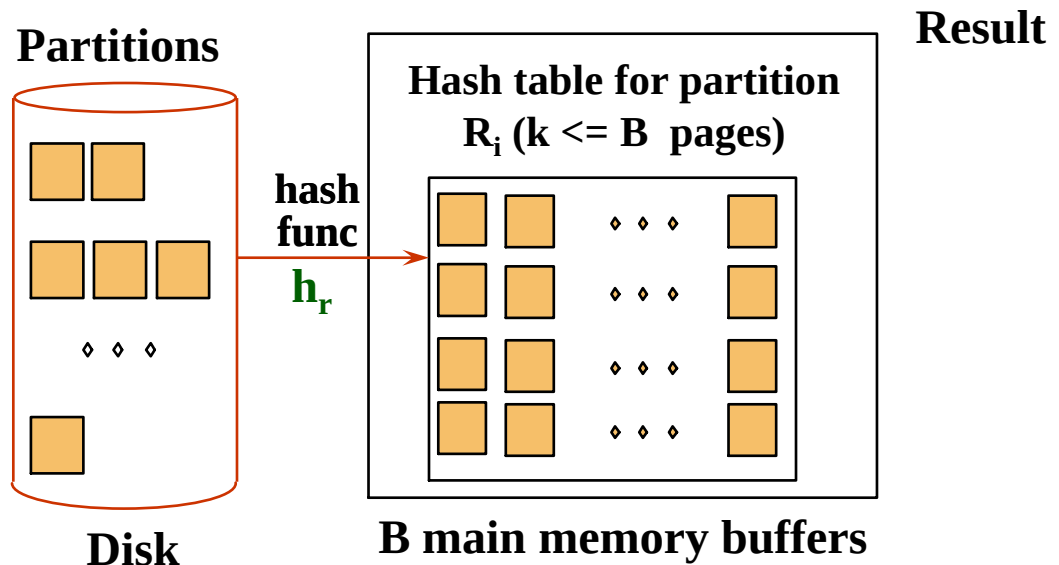


# Two Phases

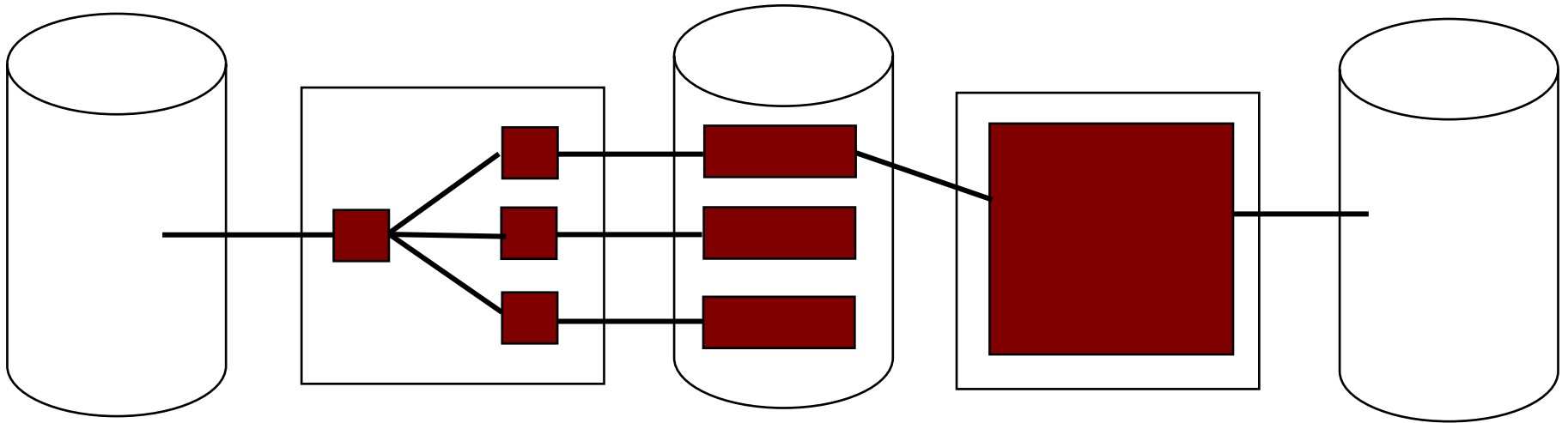
□ Partition:



□ Rehash:



# Cost of External Hashing



$$\text{cost} = 4 * N \text{ IO's}$$

# Memory Requirement

- How big of a table can we hash in two passes?
  - B-1 “partitions” result from Phase 0
  - Each should be no more than B pages in size
  - Answer:  $B(B-1)$ .
    - ▶ We can hash a table of size N pages in about  $\sqrt{N}$  space
  - Note: assumes hash function distributes records evenly!
- Have a bigger table? Recursive partitioning!
  - How many times?
    - ▶ Until every partition fits in memory !! ( $\leq B$ )



**How does this compare with  
external sorting?**



# So which one is better?

## □ Simplest analysis:

- Same memory requirement for 2 passes
- Same I/O cost
- But we can dig a bit deeper...

## □ Sorting pros:

- Great if input already sorted (or almost sorted) w/heapsort
- Great if need output to be sorted anyway
- Not sensitive to “data skew” or “bad” hash functions

## □ Hashing pros:

- For duplicate elimination, scales with # of unique values
  - ▶ Not # of values (rows)! We'll see this again.



# Summing Up Part 1

- Unordered collection model
- Read in chunks to avoid fixed I/O costs
- Patterns for Big Data
  - Streaming
  - Divide & Conquer
  - also Parallelism (but we didn't cover this here)



# Summary Part 2

- ❑ Sort/Hash Duality
  - ❑ Sorting is Conquer & Merge
  - ❑ Hashing is Divide & Conquer
- ❑ Sorting is overkill if all we want is to locate matching records
  - ❑ But sometimes a win anyhow
- ❑ Sorting sensitive to internal sort alg
  - ❑ Quicksort vs. HeapSort
  - ❑ In practice, QuickSort tends to be used
- ❑ Don't forget double buffering (with threads)



# Review – Relational Operations

- We will consider how to implement:
  - Selection (  $\sigma$  ) Selects a subset of rows from relation.
  - Projection (  $\pi$  ) Deletes unwanted columns from relation.
  - Join (  $\bowtie$  ) Allows us to combine two relations.
  - Set-difference (  $-$  ) Tuples in reln. 1, but not in reln. 2.
  - Union (  $\cup$  ) Tuples in reln. 1 and in reln. 2.
  - Also: Aggregation (SUM, MIN, etc.) and GROUP BY
- Since each op returns a relation, ops can be *composed*! After we cover the operations, we will discuss how to *optimize* queries formed by composing them.



# Selection (filter) Operators



# Schema for Examples

Sailors (*sid*: integer, *sname*: string, *rating*: integer, *age*: real)

Reserves (*sid*: integer, *bid*: integer, *day*: date, *rname*: string)

- Assume pages of 4000 bytes each
- Reserves:
  - Each tuple is 40 bytes long, 100 tuples per page, 1000 pages. (100K reservations)
- Sailors:
  - Each tuple is 50 bytes long, 80 tuples per page, 500 pages. (40K sailors)



# Simple Selections

```
SELECT *  
FROM   Reserves R  
WHERE  R.date > '1/1/2015'
```

- Of the form  $S_{R.attr \text{ op value}}(R)$
- Question: how best to perform?
  - what indexes/access paths are available?
  - what is the expected size of the result (in terms of number of tuples and/or number of pages)?
- **Size of result** approximated as
  - size of R \* reduction factor*
  - “reduction factor” is usually called *selectivity*.
  - estimate of reduction factors is based on statistics – we will discuss shortly.
    - ▶ Active research area!



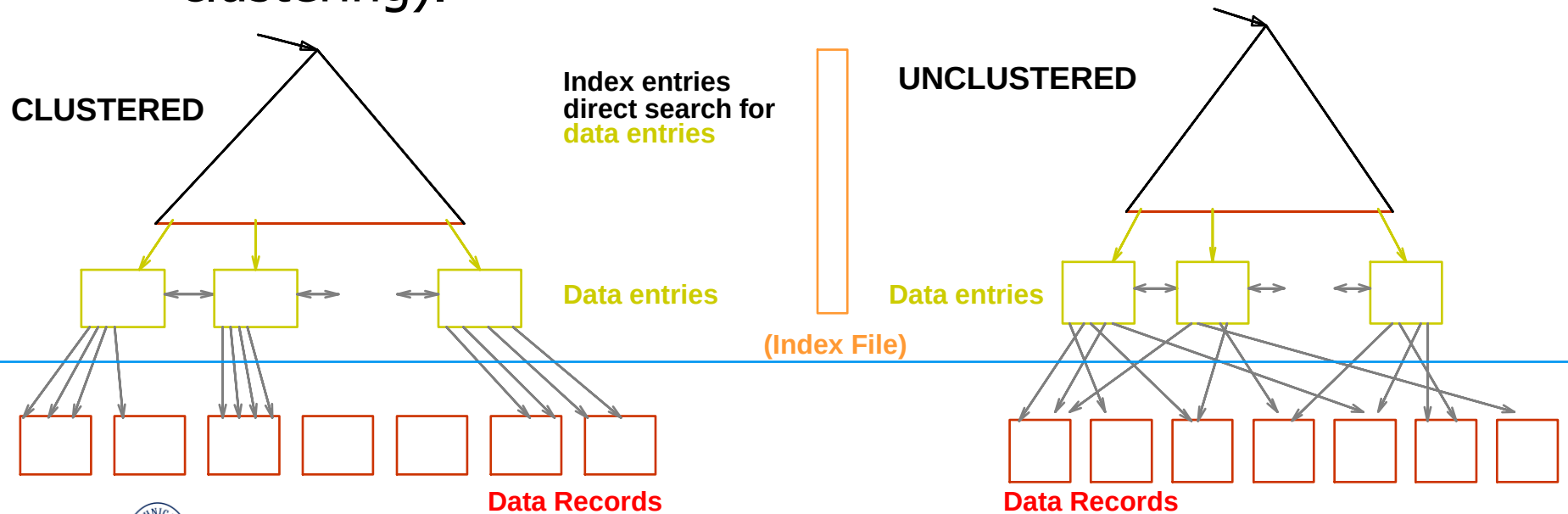
# Alternatives for Simple Selections

- With no index, unsorted:
    - Must essentially scan the whole relation
    - cost is  $M$  (#pages in  $R$ ). For “Reserves” = 1000 I/Os.
  - With no index, but sorted:
    - cost of binary search + number of pages containing results.
    - For reserves = 10 I/Os +  $\lceil \text{selectivity} * 1000 \rceil$
  - With an index on selection attribute:
    - Use index to find qualifying data entries,
    - then retrieve corresponding data records.
    - (Hash index useful only for equality selections.)
- 
- Reserves:
    - Each tuple is 40 bytes long, 100 tuples per page, 1000 pages. (100K reservations)
  - Sailors:
    - Each tuple is 50 bytes long, 80 tuples per page, 500 pages. (40K sailors)



# Using an Index for Selections

- Cost depends on #qualifying tuples and clustering.
  - Cost:
    - ▶ finding qualifying data entries (typically small)
    - ▶ plus cost of retrieving records (could be large w/o clustering).



# Using an Index for Selections

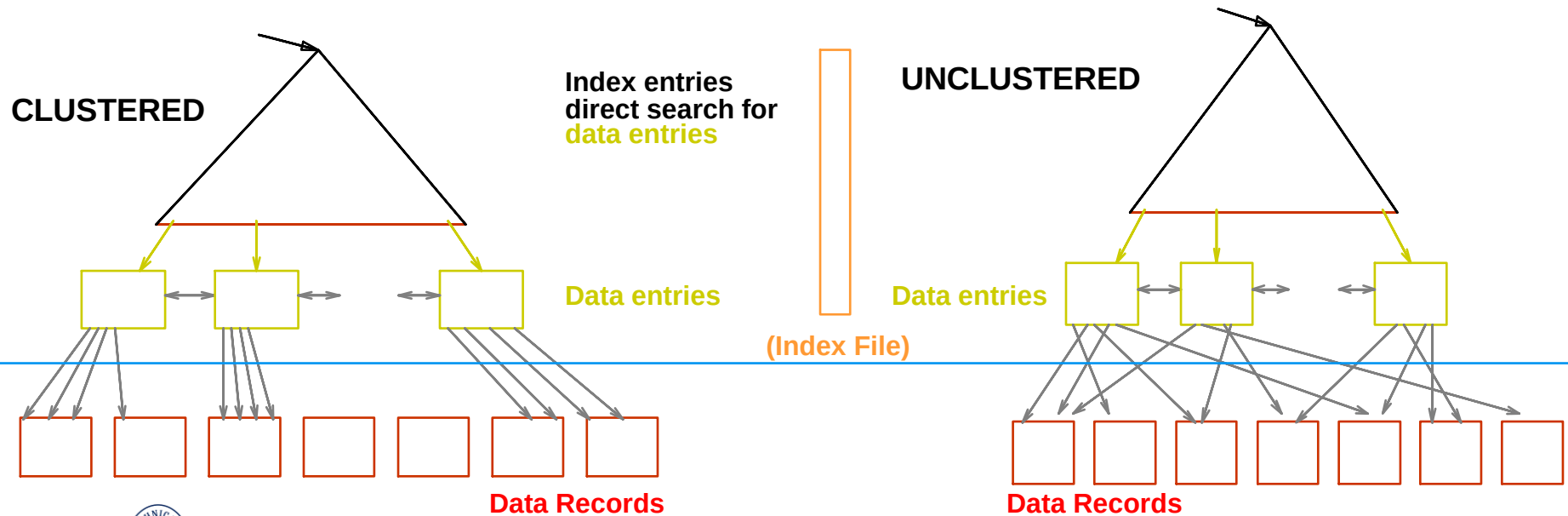
- In example “Reserves” relation, if 10% of tuples qualify (result size estimate: 100 pages, 10000 tuples).
  - ▶ With a *clustered* index, cost is little more than **100** I/Os;
  - ▶ if *unclustered*, could be more than **10000** I/Os! unless...
- Reserves:
  - Each tuple is 40 bytes long, 100 tuples per page, 1000 pages. (100K reservations)
- Sailors:
  - Each tuple is 50 bytes long, 80 tuples per page, 500 pages. (40K sailors)



# Selections using Index (cont)

## □ *Important refinement for unclustered indexes:*

1. Find qualifying data entries.
2. Sort the rid's of the data records to be retrieved.
3. Fetch rids in order. This ensures that each data page is looked at just once (though # of such pages likely to be higher than with clustering).



# General Selection Conditions

- $(day < 8/9/94 \text{ AND } rname = 'Paul') \text{ OR } bid = 5 \text{ OR } sid = 3$
- Such selection conditions are first converted to conjunctive normal form (CNF):
  - $(day < 8/9/94 \text{ OR } bid = 5 \text{ OR } sid = 3) \text{ AND } (rname = 'Paul' \text{ OR } bid = 5 \text{ OR } sid = 3)$
- A **B-tree** index matches (a conjunction of) terms that involve only attributes in a *prefix* of the search key.
  - Index on  $\langle a, b, c \rangle$  matches  $a = 5 \text{ AND } b = 3$ , but not  $b = 3$ .
- For **Hash** index, must have all attributes in search key
- Discussion of the case with Ors → (next slide).



# Algorithms for Complex Selections

**Disjunction:**  $\sigma_{\theta_1 \vee \theta_2 \vee \dots \vee \theta_n}(r)$ .

- **disjunctive selection by union of identifiers**

- Applicable if *all* conditions have available indices.
  - ▶ Otherwise use linear scan.
- Use corresponding index for each condition, and take union of all the obtained sets of record pointers.
- Then fetch records from file

- **Negation:**  $\sigma_{\neg\theta}(r)$

- Use linear scan on file
- If very few records satisfy  $\neg\theta$ , and an index is applicable to  $\theta$ 
  - ▶ Find satisfying records using index and fetch from file

